

3.04 Qualité de Code

Résumé pattern Etat



Sommaire

1.Présentation du Pattern - p1

2.Exemple d'un besoin – p2 - p5

2.1. Bad Code

2.2. Problème

2.3. Solution

3.Diagramme de Classe – p5 – p6

4.Détails des classes – p6

5.Lien avec SOLID – p7

6.Limites du pattern – p7

1. Présentation du Pattern

Le pattern État (State) est un modèle comportemental qui permet à un objet de modifier son comportement lorsque son état interne change, sans recourir à de longues structures conditionnelles (if ou switch). L'idée est de déléguer les comportements spécifiques à des classes représentant les différents états possibles de l'objet. Ce pattern rend le code plus clair, plus extensible et facilite la gestion de la complexité liée aux transitions d'états.

2. Exemple d'un besoin

Nous prendrons l'exemple de la Pizza énoncé lors de la présentation pour illustrer cette partie.

Bad Code

La Classe Pizza possède cette méthode :

```
public boolean bake() {  
    if (state == COOKED) {  
        state = BAKED;  
    } else if (state == BAKED) {  
        return false; // Can't bake a pizza already baked !  
    } else if (state == DELIVERED) {  
        return false; // Can't bake a pizza already delivered !  
    }  
    return true;  
}
```

Problème

On constate qu'il y a une cascade de **if / else** qui devra être modifiée lorsqu'on devra ajouter des états à la machine. Cela rentre en conflit avec le principe OCP.

De plus la Classe réalisant toutes les actions on en conclut qu'elle ne respecte pas non plus le principe SRP.

Solution

Pour cela nous mettons en place le Design pattern état qui s'applique comme tel :

Classe Pizza :

```
public class Pizza {  
    private String name;  
    private PizzaState state;  
  
    public Pizza(String name) {  
        this.name = name;  
        this.state = new CookedState(); // État initial par défaut  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public PizzaState getState() {  
        return state;  
    }  
  
    public void setState(PizzaState state) {  
        this.state = state;  
    }  
  
    public boolean bake() {  
        return state.bake(this);  
    }  
  
    public boolean deliver() {  
        return state.deliver(this);  
    }  
}
```

9

Interface PizzaState :

```
public interface PizzaState {  
    boolean bake(Pizza pizza);  
    boolean deliver(Pizza pizza);  
}
```

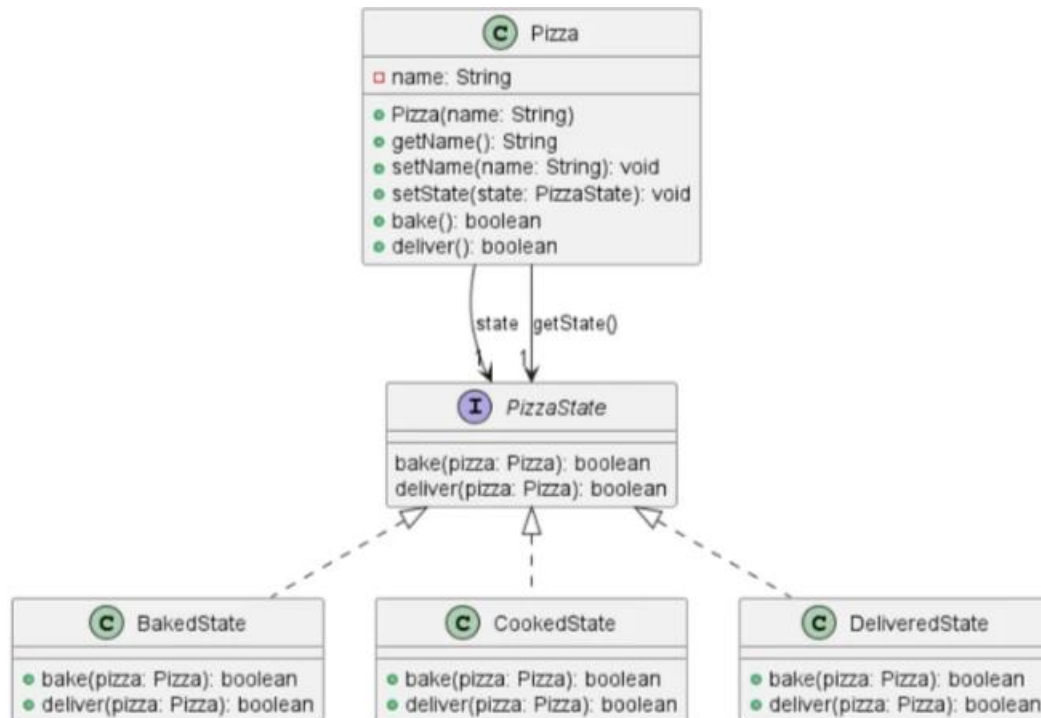
Classe CookedState qui implémente PizzaState

HUMAYOU Thibault
GENESTE DESETTE Thomas
FAVRE-ANDRE Gabriel

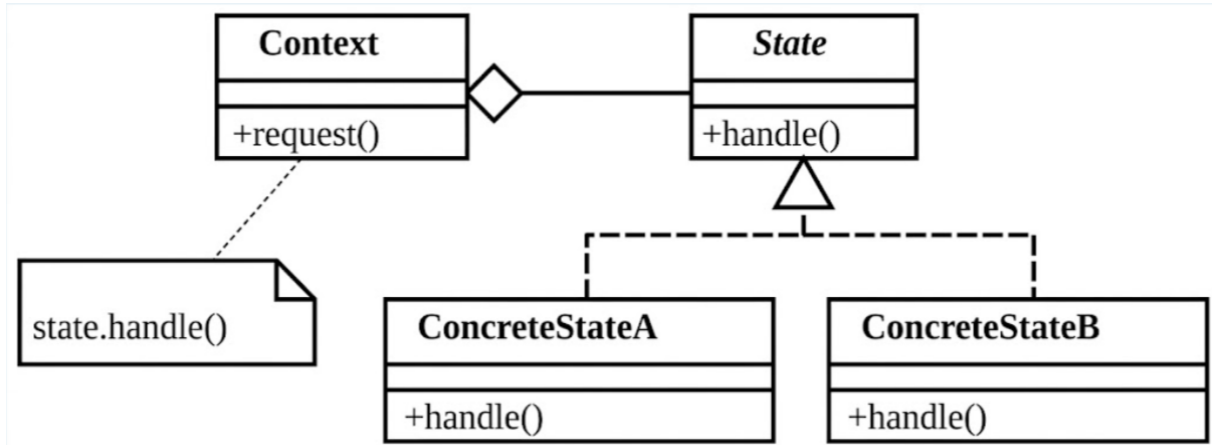
```
public class CookedState implements PizzaState {  
  
    @Override  
    public boolean bake(Pizza pizza) {  
        System.out.println("Cuisson de la pizza en cours...");  
        pizza.setState(new BakedState());  
        return true;  
    }  
  
    @Override  
    public boolean deliver(Pizza pizza) {  
        System.out.println("Erreur : Impossible de livrer une pizza pas encore cuite !");  
        return false;  
    }  
}
```

3. Diagramme de Classe

A partir de toutes ces propositions issues du pattern état nous pouvons établir un diagramme de classe qui est le suivant :



Ce dernier nous permet d'en tirer un diagramme de Classe générique du pattern en suivant les mêmes règles qui est le suivant :



Ce Diagramme nous servira donc à l'avenir lors de nos conception qui demande un certains nombres d'état puisque nous le citerons plus tard mais il possède également des limites.

4. Détails des classes

Nom Générique	Description
Context	élément principal qui possède l'état. il peut être changé grâce au setter
State	Interface commune qui permet aux ConcreteState d'être instancié dans Context. Il possède les méthodes et sert de passerelle
ConcreteState	Classe qui implémente l'interface afin de donner accès aux méthodes nécessaires à la Classe dont elle est l'attribut.

5. Lien avec SOLID

Le pattern état respecte bien les principes SOLID de la manière suivantes :

- **SRP** : les comportements sont séparés dans plusieurs classes (Les ConcreteState) afin que seuls les états qui doivent posséder ses comportements les possèdent
- **OCP** : Le code peut être étendu sans devoir modifier les classes précédemment établies.
- **LSP** : les ConcreteState sont utilisées à la place de l'interface State.
- **ISP** : Les interfaces sont compartimentées, elles n'utilisent que ce qui est nécessaire aux états qui en découlent. Les interfaces sont séparées. L'ISP est respecté
- **DIP** : Les classes ConcreteState ainsi que la classe qui possède un état dépendent d'abstraction ou d'interface (la classe état), le DIP est donc respecté

6. Limites du pattern

Les limites du pattern sont les suivantes :

- Dans un code qui ne compte que peu d'états probables la mise en place du pattern demandera trop de ressources et complexifiera la tâche pour peu de résultats.
- De plus cela augmente considérablement le nombre de classe utilisées puisque chaque état concret a sa classe associée. Cela implique donc qu'en cas de maintenance le travail sera nettement plus laborieux ce qui implique des coûts plus élevés.